

Task 1: To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

```
#include <stdio.h>
#include <string.h>

int main() {
    char input[100];

    while (1) {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL) {
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0) {
            printf("Exiting shell...\n");
            break;
        }

        if (strlen(input) == 0) {
            continue;
        }

        printf("You entered: %s\n", input);
    }

    return 0;
}
```

```
ramya@LAPTOP-OPDAF309: ~$ mkdir skill3
ramya@LAPTOP-OPDAF309: ~$ cd skill3
ramya@LAPTOP-OPDAF309: ~/skill3$ nano task1.c
ramya@LAPTOP-OPDAF309: ~/skill3$ gcc task1.c -o task1
ramya@LAPTOP-OPDAF309: ~/skill3$ ./task1
myshell> hello
You entered: hello
myshell> database
You entered: database
myshell> exit
Exiting shell...
ramya@LAPTOP-OPDAF309: ~/skill3$ |
```

Task 2: Using the following system calls, copy the content from File1.txt to File2.txt.

1. **Open()**
2. **Read()**
3. **Write()**
4. **Close()**

open()

```

#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    int fd1, fd2;
    char buffer[100];
    ssize_t n;

    fd1 = open("File1.txt", O_RDONLY);

    if (fd1 == -1) {
        perror("Error opening File1.txt");
        return 1;
    }

    fd2 = open("File2.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);

    if (fd2 == -1) {
        perror("Error opening File2.txt");
        close(fd1);
        return 1;
    }

    while ((n = read(fd1, buffer, sizeof(buffer))) > 0) {
        if (write(fd2, buffer, n) != n) {
            perror("Error writing to File2.txt");
            close(fd1);
            close(fd2);
            return 1;
        }
    }
}

```

```

    if (n == -1) {
        perror("Error reading File1.txt");
    }

    close(fd1);
    close(fd2);

    printf("File copied successfully.\n");

    return 0;
}

```

```
ramya@LAPTOP-OPDAF309:~/skill3$ echo "This is the content of File1." > File1.txt
ramya@LAPTOP-OPDAF309:~/skill3$ cat File1.txt
This is the content of File1.
ramya@LAPTOP-OPDAF309:~/skill3$ nano task2.c
ramya@LAPTOP-OPDAF309:~/skill3$ gcc task2.c -o task2
ramya@LAPTOP-OPDAF309:~/skill3$ ./task2
File copied successfully.
ramya@LAPTOP-OPDAF309:~/skill3$ cat File2.txt
This is the content of File1.
ramya@LAPTOP-OPDAF309:~/skill3$ cmp File1.txt File2.txt
ramya@LAPTOP-OPDAF309:~/skill3$ diff File1.txt File2.txt
```

Task 3: To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction.

```
#include <stdio.h>
#include <unistd.h>
#include <termios.h>
#include <string.h>

int main() {
    struct termios oldt, newt;
    char buffer[100];
    int index = 0;
    char ch;

    tcgetattr(STDIN_FILENO, &oldt);
    newt = oldt;

    newt.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);

    printf("myshell> ");
    fflush(stdout);

    while (1) {
        read(STDIN_FILENO, &ch, 1);

        if (ch == '\n') {
            buffer[index] = '\0';

            printf("\nYou entered: %s\n", buffer);

            if (strcmp(buffer, "exit") == 0) {
                break;
            }

            index = 0;
            printf("myshell> ");
            fflush(stdout);
        }
    }
}
```

```

        else if (ch == 127 || ch == 8) {
            if (index > 0) {
                index--;
                printf("\b \b");
                fflush(stdout);
            }
        }
        else if (index < 99) {
            buffer[index++] = ch;
            write(STDOUT_FILENO, &ch, 1);
        }
    }

    tcsetattr(STDIN_FILENO, TCSANOW, &oldt);

    return 0;
}

```

```

ramya@LAPTOP-OPDAF309:~/skill3$ nano task3.c
ramya@LAPTOP-OPDAF309:~/skill3$ gcc task3.c -o task3
ramya@LAPTOP-OPDAF309:~/skill3$ ./task3.c
-bash: ./task3.c: Permission denied
ramya@LAPTOP-OPDAF309:~/skill3$ ./task3
myshell> hello
You entered: hello
myshell> database
You entered: database
myshell> management
You entered: management
myshell> system
You entered: system
myshell> exit
You entered: exit
ramya@LAPTOP-OPDAF309:~/skill3$

```

Task 4: To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality

```
#include <stdio.h>
#include <unistd.h>
#include <termios.h>
#include <string.h>

#define MAX_HISTORY 10
#define MAX_INPUT 100

int main() {
    struct termios oldt, newt;
    char history[MAX_HISTORY][MAX_INPUT];
    int history_count = 0;
    int history_pos = 0;

    char input[MAX_INPUT];
    int index = 0;
    char ch;

    tcgetattr(STDIN_FILENO, &oldt);
    newt = oldt;
    newt.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);

    while (1) {
        printf("myshell> ");
        fflush(stdout);

        index = 0;
        history_pos = history_count;

        while (1) {
            read(STDIN_FILENO, &ch, 1);

            if (ch == '\n') {
                input[index] = '\0';
                printf("\n");
            }
        }
    }
}
```

```

    if (strcmp(input, "exit") == 0) {
        tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
        return 0;
    }

    if (index > 0) {
        if (history_count < MAX_HISTORY) {
            strcpy(history[history_count], input);
            history_count++;
        } else {
            for (int i = 0; i < MAX_HISTORY - 1; i++) {
                strcpy(history[i], history[i + 1]);
            }
            strcpy(history[MAX_HISTORY - 1], input);
        }
    }

    printf("Command: %s\n", input);
    break;
}

else if (ch == 127 || ch == 8) {
    if (index > 0) {
        index--;
        printf("\b \b");
        fflush(stdout);
    }
}

else if (ch == 27) {
    char seq[2];

    if (read(STDIN_FILENO, &seq[0], 1) &&
        read(STDIN_FILENO, &seq[1], 1)) {

        if (seq[0] == '[' && seq[1] == 'A') {

```



```

        if (history_pos > 0) {
            history_pos--;

            while (index > 0) {
                printf("\b \b");
                index--;
            }

            strcpy(input, history[history_pos]);
            index = strlen(input);
            printf("%s", input);
            fflush(stdout);
        }

    else if (seq[0] == '[' && seq[1] == 'B') {
        if (history_pos < history_count - 1) {
            history_pos++;

            while (index > 0) {
                printf("\b \b");
                index--;
            }

            strcpy(input, history[history_pos]);
            index = strlen(input);
            printf("%s", input);
            fflush(stdout);
        }
    }
}

```

```

    else if (index < MAX_INPUT - 1) {
        input[index++] = ch;
        write(STDOUT_FILENO, &ch, 1);
    }
}

tcsetattr(STDIN_FILENO, TCSANOW, &oldt);

return 0;
}

```

```
ramya@LAPTOP-OPDAF309:~/skill3$ gcc task4.c -o task4
ramya@LAPTOP-OPDAF309:~/skill3$ ./task4
myshell> hello
Command: hello
myshell> database
Command: database
myshell> hello
Command: hello
myshell> exit
```

Task 5: To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node {
    char *command;
    struct Node *next;
} Node;

int main() {
    char *buffer;
    size_t size = 10;
    size_t length = 0;
    int ch;

    Node *head = NULL;
    Node *tail = NULL;

    buffer = malloc(size);

    if (buffer == NULL) {
        perror("malloc");
        return 1;
    }

    printf("Enter a command: ");

    while ((ch = getchar()) != '\n' && ch != EOF) {
        if (length + 1 >= size) {
            size *= 2;

            char *temp = realloc(buffer, size);

            if (temp == NULL) {
                free(buffer);
                perror("realloc");
                return 1;
            }
        }
    }

```

```

        }

        buffer = temp;
    }

    buffer[length++] = ch;
}

buffer[length] = '\0';

printf("Command entered: %s\n", buffer);

Node *new_node = malloc(sizeof(Node));

if (new_node == NULL) {
    free(buffer);
    perror("malloc");
    return 1;
}

new_node->command = malloc(strlen(buffer) + 1);

if (new_node->command == NULL) {
    free(new_node);
    free(buffer);
    perror("malloc");
    return 1;
}

strcpy(new_node->command, buffer);
new_node->next = NULL;

head = new_node;
tail = new_node;

printf("Command stored in linked list: %s\n", tail->command);

```

```

    free(buffer);

    Node *current = head;

    while (current != NULL) {
        Node *next = current->next;
        free(current->command);
        free(current);
        current = next;
    }

    printf("Memory released successfully.\n");

    return 0;
}

```

```

ramya@LAPTOP-OPDAF309:~/skill3$ nano task5.c
ramya@LAPTOP-OPDAF309:~/skill3$ gcc -Wall -Wextra task5.c -o task5
ramya@LAPTOP-OPDAF309:~/skill3$ ./task5
Enter a command: This is a long command for testing dynamic buffer allocation
Command entered: This is a long command for testing dynamic buffer allocation
Command stored in linked list: This is a long command for testing dynamic buffer allocation
Memory released successfully.

```

Verify Using Valgrind :

```

ramya@LAPTOP-OPDAF309:~/skill3$ gcc -Wall -Wextra task5.c -o task5
ramya@LAPTOP-OPDAF309:~/skill3$ valgrind --leak-check=full --track-origins=yes ./task5
==1324== Memcheck, a memory error detector
==1324== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==1324== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==1324== Command: ./task5
==1324==
Enter a command: Hello valgrind
Command entered: Hello valgrind
Command stored in linked list: Hello valgrind
Memory released successfully.
==1324==
==1324== HEAP SUMMARY:
==1324==   in use at exit: 0 bytes in 0 blocks
==1324==   total heap usage: 6 allocs, 6 frees, 2,109 bytes allocated
==1324==
==1324== All heap blocks were freed -- no leaks are possible
==1324==
==1324== For lists of detected and suppressed errors, rerun with: -s
==1324== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```